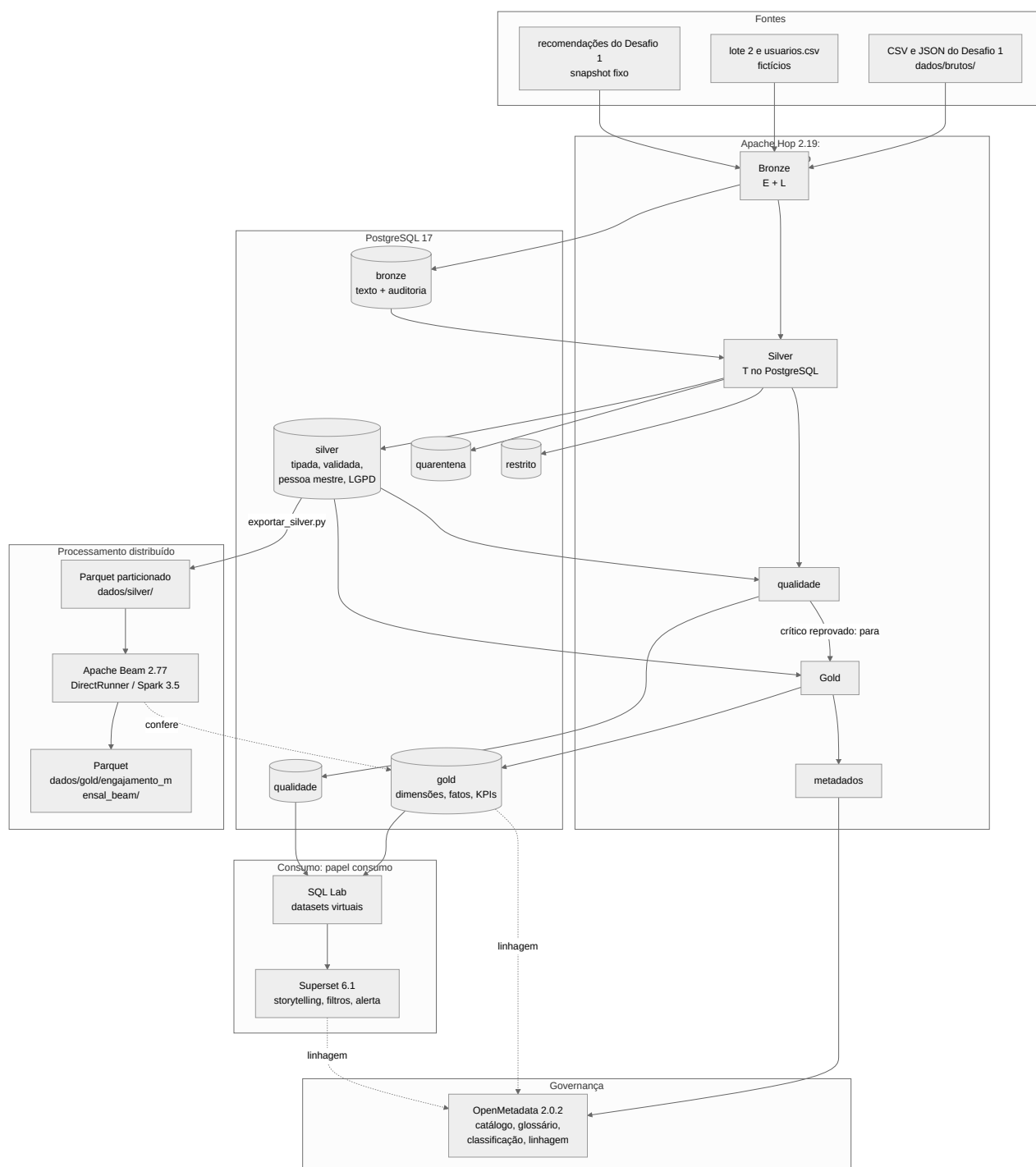


Arquitetura do Desafio 2 (RF19)

A solução do Desafio 1 era um script Python que lia os arquivos, validava, gravava e calculava tudo numa execução só. O Desafio 2 transforma esse fluxo numa plataforma em camadas, orquestrada, governada e auditável. O Desafio 1 continua funcionando e passa a ser uma das **fontes**.

Tudo roda em Docker Compose (Linux, macOS Intel e Apple Silicon, Windows com WSL2), sem instalar nada além do Docker.

Fluxo de ponta a ponta



Onde acontecem extração, transformação e carga

Etapa	E / T / L	Ferramenta	O que faz
Fontes → Bronze	E + L	Hop (<code>bronze_*.hpl</code>)	lê CSV e JSON e grava como texto, sem transformar, com <code>_execucao_id</code> , <code>_origem</code> , <code>_linha</code> e <code>_ingerido_em</code> ; append-only
Bronze → Silver	T	SQL no PostgreSQL (<code>validacao.classificar_*</code>), roteado pelo Hop (<code>silver_*.hpl</code>)	tipagem, padronização, validação, deduplicação, dados mestres e proteção LGPD; rejeitados → quarentena; publicação atômica
Silver → qualidade	T (verificação)	SQL (<code>qualidade.executar</code>)	8 testes; um crítico reprovado bloqueia a Gold
Silver → Gold	T	SQL (<code>gold.publicar</code>)	dimensões, fatos e KPIs, numa transação
Silver → Parquet → Beam	E, T, L	<code>exportar_silver.py</code> + Apache Beam	extrai a Silver, transforma fora do banco (Beam) e carrega em Parquet
Gold → Superset	consumo	SQL Lab e Superset, papel consumo	consultas, datasets virtuais, dashboards e alerta
todas → OpenMetadata	metadados	ingestões do OpenMetadata, disparadas pelo Hop	catálogo e linhagem atualizados a cada execução

Classificação: ELT no fluxo principal, arquitetura híbrida no todo

- **Fluxo principal: ELT.** O dado é extraído e carregado bruto na Bronze. Toda transformação acontece depois, dentro do PostgreSQL, em SQL versionado, e o Hop só orquestra e roteia.
- **Ramo analítico: ETL.** O Beam extrai a Silver para Parquet, transforma fora do banco e carrega o resultado em arquivos. Esse ramo existe para o processamento distribuído (RF24, RF25) e é conferido contra a Gold.

Por isso a arquitetura é **híbrida**, com o ELT como espinha dorsal.

Justificativa

Critério	Por que ELT no fluxo principal
Governança	a Bronze guarda o dado original de toda execução, e cada linha sabe de onde veio (arquivo, linha, execução). Qualquer número do dashboard pode ser rastreado até a linha da fonte (<code>linhagem.md</code>). As regras estão num lugar só (<code>sql/silver.sql</code>), versionadas e legíveis
Reprocessamento	corrigir uma regra ou um registro da quarentena e reprocessar a Silver não exige reler a fonte: a Bronze da execução está lá. A quarentena tem identidade estável, e a correção substitui o registro da Bronze na próxima Silver
Desempenho	as regras que cruzam fontes (o usuário existe? a interação é anterior à publicação?) são junções que o banco resolve de uma vez. Num ETL no Hop, cada verificação seria um lookup linha a linha na memória da ferramenta
Custo	um único PostgreSQL faz o armazenamento e a transformação. Não há outro motor para operar no fluxo principal, e o volume, milhares de registros por lote, cabe folgado nele
Consistência	a Silver e a Gold são publicadas em transações únicas. Quem consulta vê a versão anterior inteira ou a nova inteira, nunca uma carga parcial (RF23)

Critério	Por que ETL no ramo Beam
Escala	o dia em que o volume de interações não couber num banco relacional, o cálculo mensal já existe em Beam, sobre Parquet, e roda num cluster Spark sem mudar o código (RF25)
Custo de leitura	o Parquet particionado por mês lê só o mês e as colunas pedidos: 11 vezes mais rápido que o JSON e 4 vezes mais que o CSV no volume de 200 mil (parquet_beam.md)
Sem divergência	o resultado do Beam é comparado com a Gold a cada execução (<code>igual_a_gold</code>)

Limitações da solução anterior (Desafio 1, scripts isolados)

No Desafio 1	Consequência	No Desafio 2
o dado bruto não era guardado; o script lia, tratava e gravava o resultado	não dava para auditar nem reprocessar sem a fonte original	Bronze append-only, com auditoria por linha
um único script, sem etapas registradas	uma falha no meio só aparecia no log; não se sabia o que tinha rodado	<code>controle.execucao</code> e <code>controle.etapa</code> , com início, fim, contagens e status de cada etapa; execução isolada de uma etapa
rejeitados num arquivo JSON	não havia como corrigir e reprocessar um registro	quarentena com ciclo pendente → corrigido → reprocessado, ou descartado
carga direto nas tabelas finais	uma falha no meio deixava os bancos parcialmente atualizados	preparo + publicação atômica
nenhum teste de qualidade separado da validação	um lote ruim chegava ao dashboard	8 testes; crítico reprovado bloqueia a Gold
sem catálogo nem linhagem	o significado dos KPIs estava só na documentação	OpenMetadata com glossário, donos, classificação e linhagem
sem fonte de usuários; nenhuma proteção de dado pessoal	—	cadastro fictício, pseudonimização, hash com salt, mascaramento, papel <code>consumo</code>
execução manual	—	agendamento nativo do Hop (<code>hop-agendador</code>)

Componentes e versões (RF15)

Componente	Versão	Imagem	Perfil do Compose
PostgreSQL + pgvector	17 + 0.8	<code>pgvector/pgvector:pg17</code>	base
MongoDB (Desafio 1)	8.2	<code>mongo:8.2</code>	base
Apache Superset	6.1.0	<code>desafio-superset</code> (Dockerfile próprio)	base
Redis, Mailpit (alertas)	7.4, 1.31.3	<code>redis:7.4-alpine</code> , <code>axllent/mailpit:v1.31.3</code>	alertas
Apache Hop	2.19.0	<code>apache/hop</code> , <code>apache/hop-web</code>	<code>pipeline</code> , <code>hop</code> , <code>agendamento</code>
Apache Beam (SDK Python e job server)	2.77.0	<code>apache/beam_python3.12_sdk</code> (+ <code>psycopg</code>), <code>apache/beam_spark3_job_server</code>	<code>pipeline</code> , <code>beam</code>
Apache Spark	3.5.0	<code>apache/spark:3.5.0-scala2.12-java11-ubuntu</code>	<code>beam</code>
OpenMetadata (servidor e ingestão)	2.0.2	<code>docker.getcollate.io/openmetadata/*</code>	governanca
Elasticsearch	9.3.0	<code>docker.elastic.co/elasticsearch/elasticsearch</code>	governanca

Componente	Versão	Imagem	Perfil do Compose
Playwright (capturas e PDFs)	1.59.0	mcr.microsoft.com/playwright/python	pipeline

Memória. Com tudo no ar, a plataforma pede 10 a 12 GB para o Docker:

Parte	Memória
base	~0,7 GB
governança	~3,5 GB
Spark	~1,3 GB, mais até 2 GB por job
Superset (~0,5 GB) com o perfil alertas (~0,5 GB)	~1 GB

Com 8 GB, suba um perfil por vez (`docker compose --profile <perfil> stop` libera). Durante o desenvolvimento, uma VM de 8 GB com outros projetos no ar matou processos por falta de memória. Por isso todos os serviços têm `restart: unless-stopped`.

Configuração e segredos (RF15)

O quê	Onde
senhas, LGPD_SALT, LGPD_CHAVE_PSEUDONIMO, papel consumo, token do OpenMetadata	só no <code>.env</code> (no <code>.gitignore</code>); modelo em <code>.env.example</code>
parâmetros não sensíveis do Python (fontes, Parquet, runners do Beam)	<code>config.yaml</code>
parâmetros do Hop	<code>hop/environments/docker.json</code> ; os segredos chegam por um segundo arquivo de ambiente gerado dentro do container (<code>docker/hop/segredos.sh</code>)
regras, testes, Gold, descrições	SQL versionado em <code>sql/</code> , aplicado pelo <code>db-init</code> (idempotente, não destrutivo)
Superset e OpenMetadata	configurados por código (<code>superset/provisionar.py</code> , <code>openmetadata/provisionar.py</code>), idempotente

Ordem de execução

```

cp .env.example .env                                # preencher as senhas e os dois segredos LGPD
docker compose up -d                                # base: PostgreSQL, MongoDB, Superset, db-init
docker compose --profile governanca up -d            # OpenMetadata (a primeira subida demora)
docker compose run --rm beam openmetadata/provisionar.py # serviços, ingestões e governança
docker compose run --rm hop workflows/principal.hwf   # Bronze → Silver → qualidade → Gold →
metadados
docker compose run --rm beam beam/exportar_silver.py --volume
docker compose run --rm beam beam/pipeline.py --runner direct
docker compose --profile beam up -d && docker compose run --rm beam beam/pipeline.py --runner spark
docker compose run --rm beam superset/provisionar.py  # datasets, dashboards e alerta

```

A seção Desafio 2 do [README](#) tem o passo a passo completo, com as evidências e as demonstrações.

Documentos por requisito

Requisito	Documento
RF15, RF19	este documento
RF16–RF18	storytelling.md
RF20–RF23	pipeline_hop.md , contratos.md
RF24, RF25	parquet_beam.md
RF26	camada_gold.md
RF27, RF28, RF30	governanca.md
RF29	linhagem.md
RF31	../qualidade/regras.md
RF32, RF33	../lgpd/inventario_de_dados.md , ../lgpd/tecnicas_de_protecao.md